The Future of Programming:

From Problem Specifications to AI-Compiled Implementations

—

A Type-Theoretic Foundation

Ashwin Rao
Adjunct Professor, ICME, Stanford University

March 15, 2026

Abstract

The software industry has spent seventy years getting progressively better at describing *how* to implement solutions, while remaining remarkably imprecise about describing *what problem* is actually being solved. This essay argues that we are on the verge of a fundamental correction: driven by the capabilities of modern AI, the act of programming will shift from writing implementation code to writing formal, structured specifications of problems. We trace the historical and theoretical roots of this shift — from Church's lambda calculus and the Curry–Howard correspondence to modern dependent type theory — and propose a realistic two-stage compilation pipeline in which AI transforms human problem descriptions into formal typed specifications, and then into verified implementations. The intended audience is anyone with a working familiarity with computer science and modern AI.

The Future of Programming

We are on the verge of a more fundamental shift in programming than most people in the industry currently appreciate. Not just better tools, faster code generation, or smarter autocomplete — but a change in what programming *is*.

For most of the history of software, brilliant engineers have dived into *how* to build something before rigorously nailing down *what problem* they were actually solving. This is not simply a matter of professional discipline. It is a deeper, structural problem: a seventy-year architectural choice baked into how the industry was built. Understanding why requires a short detour into the theoretical foundations of computing.

Two Formalisms, One Fork in the Road

In the 1930s, two foundational models of computation emerged almost simultaneously, and they encoded fundamentally different habits of mind.

Alan Turing's machine described computation as a sequence of mechanical state transitions — an irreducibly *how*-oriented formalism. To specify a computation in Turing's framework is to describe, in meticulous low-level detail, every step the machine must take.

Alonzo Church's lambda calculus described computation as the composition of mathematical functions — a *what*-oriented formalism. To specify a computation in Church's framework is to define what the function *is*: its inputs, its outputs, and the precise relationship between them.

The Church–Turing thesis established that these two formalisms are computationally equivalent — anything expressible in one is expressible in the other. But this equivalence conceals a crucial asymmetry in epistemic style. **In Church's framework, the solution specification and the problem specification are inseparable.** You cannot write a well-typed, compositional function without having already characterised precisely what problem it solves. To define the domain and codomain of a function is to formally state what you are computing and why. Turing's approach carries no such obligation: one can write a sequence of machine operations without ever articulating the problem being addressed.

Functional programming languages — Haskell being the clearest modern exemplar — carry Church's spirit: they encourage the programmer to think first in terms of *what* a computation is before considering *how* it will execute.

How the Industry Went the Other Way

Despite the theoretical elegance of Church's approach, it was Turing's model that the industry adopted, and for understandable reasons. Early computers were, in essence, physical Turing machines: sequential, imperative, memory-constrained devices. Imperative languages mapped directly onto that hardware and ran efficiently. Functional languages, being closer to mathematical abstractions, required more sophisticated compilers and more memory — both scarce resources in the early decades of computing. Once C dominated systems programming, the entire ecosystem of tooling, education, and hiring calcified around the imperative model.

The resulting historical arc is familiar to every programmer: Turing machines, then punched cards, then assembly, then C, then C++, then Java, then Python. Each generation raised the level of abstraction over *implementation details*, making programming progressively more accessible. But throughout this entire arc, the industry never fundamentally crossed over to Church's side of the ledger. Each step made it easier to

describe solutions. None of them made it more rigorous to describe problems.

The consequences have been severe. When implementation mechanics take priority over problem specification, the result is poorly designed systems, unnecessary complexity, and an ever-growing mountain of technical debt. A very large fraction of the complexity in modern software systems is *accidental* rather than *essential* — it arises not from the inherent difficulty of the problem but from the failure to understand and specify the problem rigorously before reaching for a solution.

Formal Methods: The Road Not Taken

It is worth noting that the idea of formally specifying problems before implementing solutions is not new. A research tradition of formal specification languages has existed for decades. Tools such as Z notation, VDM, TLA+, Alloy, Coq, and Agda all represent attempts to bring mathematical rigour to the problem specification side of software development.

These efforts never went mainstream. The reason is instructive: even when a formal specification existed, a human expert still had to bridge the gap between the spec and a working implementation. That gap demanded rare expertise and enormous effort, and so was never crossed at the scale the industry required. Formal methods remained a specialised discipline, confined largely to safety-critical domains such as aerospace and cryptographic protocols.

This history raises an important question: if formal specification languages have existed for decades without achieving adoption, what is different now? The answer is AI.

Why AI Changes Everything

AI can serve as the compiler between a precise problem specification and a working implementation — exactly as high-level language compilers once spared programmers from writing assembly code. The human's job is to specify *what*. The machine's job is to produce *how*. The gap that defeated formal methods for decades — the translation from spec to code — is precisely the kind of task at which modern large language models, guided by formal type systems, are increasingly capable.

This reframes the future of programming entirely. The next generation of programming will not be about writing better code. It will be about writing better *problem specifications*. We will converge on a formal language purpose-built for exactly that: a structured, precise language for describing what you want solved, which AI then compiles into a verified implementation. **This problem specification language will be the new programming language and the problem specifications we write in this language will be the new source code**.

The remainder of this paper addresses two questions. First: what is the right mathematical foundation for such a specification language? Second: given that most practitioners cannot be expected to write mathematical specifications from scratch, what does a realistic and deployable architecture look like?

This requires a bit of a detour into mathematics, and specifically into *Type Theory*.

What Is Type Theory?

Type theory, originating with Bertrand Russell and substantially developed by Per Martin-Löf in the 1970s, is a foundational system for mathematics and logic in which every object carries a *type*, and types are themselves mathematical objects that can be reasoned about. The central intuition is simple but profound:

A type is a proposition. A program that inhabits that type is a proof of that proposition.

This equivalence is known as the **Curry–Howard correspondence**, and it is one of the deepest results in theoretical computer science. It establishes a three-way isomorphism between logic, type theory, and programming:

Logic	Type Theory	Programming
Proposition	Type	Specification
Proof	Term	Program
True proposition	Inhabited type	Working implementation

This is not metaphor — it is mathematically precise. When you write a type, you state a claim about the world. When you write a program of that type, you prove that claim constructively. The direct lineage from Church's lambda calculus is clear: Church said a computation is a function; type theory says the function's type is a proposition about what that function computes; a program is the proof that the proposition is satisfiable.

A Spectrum of Expressiveness

Most programmers encounter type theory in diluted form through the type systems of their languages. There is a spectrum of expressiveness worth understanding.

Simple types

In languages such as C and Java, types classify data: `int`, `string`, `bool`. They catch basic errors but say almost nothing about behaviour. The signature

```
int add(int x, int y)
```

tells us the function accepts two integers and returns one. It says nothing about what the function actually computes.

Parametric and generic types

Languages such as Haskell and Java (with generics) allow types to be quantified over type variables: `List<T>`, `Functor f`. This enables structural correctness guarantees through abstraction — a `map` function that is polymorphic over `List` cannot, by construction, drop or reorder elements.

Dependent types

Dependent type systems — found in Coq, Agda, Lean, and Idris — are where type theory becomes a genuine specification language. In a dependently typed system, **types can depend on values**. This allows arbitrary propositions to be encoded as types.

Consider a length-indexed vector type:

$$\texttt{Vector} : \mathbb{N} \to \texttt{Type} \to \texttt{Type}$$

A value of type `Vector` n a is a list of *exactly* n elements of type a. The function

```
append : Vector m a → Vector n a → Vector (m+n) a
```

is not merely typed — it is *proven correct by construction.* An implementation returning a vector of the wrong length will not typecheck. The compiler is the verifier.

Further examples of propositions expressible as dependent types:

- `SortedList a` — a list guaranteed by its type to be sorted; no runtime assertion required.
- `{ x : Int | x > 0 }` — a refinement type encoding a positive integer; the constraint is in the type, not buried in an assertion.
- `Authenticated (Request r)` — a request type that can only be constructed after authentication has been verified; security as a type invariant.

In a fully dependently typed language, the distinction between *writing a specification* and *writing a proof* dissolves entirely. The type *is* the specification; the program *is* the evidence that the specification is satisfiable.

Type Theory as the Foundation of the Problem Spec Language

We can now make the connection to our central thesis precise. Dependent type theory provides the natural mathematical substrate for the future problem specification language. A problem specification in this framework might look as follows:

```
-- Specify the problem: sort a list
sort : (a : Type) → Ord a → (input : List a)
     → { result : List a | sorted result ∧ permutation
        result input }
```

This type asserts: given any ordered type `a` and an input list, produce a result that is *both* sorted *and* a permutation of the input. This is not a comment, a test, or a docstring — it is a mathematical proposition. Any program inhabiting this type is, by construction, a correct sorting algorithm. The typechecker does not probabilistically check correctness; it verifies it.

Non-functional requirements (NFR) — latency, throughput, security, compliance — are first-class citizens of this framework. Refinement types and dependent types encode them as typed constraints directly in the specification:

```
processPayment
  : (req : PaymentRequest)
  → Latency p95 < 50ms
  → Compliant req PCI_DSS
  → { result : PaymentResult | settled result ∨
     explicitly_failed result }
```

The same functional specification, composed with different non-functional constraint types, produces a different valid implementation space — and the AI compiler navigates that space accordingly. This directly addresses one of the most persistent sources of late-discovered complexity in software projects: non-functional requirements (NFR) that live informally in design documents and tribal knowledge rather than as first-class, verifiable constraints.

A Realistic Two-Stage Compilation Pipeline

There is an important practical objection to the vision so far. Dependent type theory is formidably inaccessible to most practitioners. Expecting engineers, domain experts, and architects to write specifications in Coq or Agda is not a realistic near-term proposition — and it would recreate the same accessibility problem that kept formal methods from going mainstream in the first place.

The resolution is that the right architecture is not a single compilation step from human intent to implementation, but **two**, with a formal typed specification serving as a crucial intermediate representation.

Stage 1: The human problem specification

The starting point is a problem description authored by a human in whatever combination of formalisms they are comfortable with. This is not a free-form chat prompt — it is a deliberate, structured artefact, composed with as much clarity and precision as the author can bring. It might include:

- **Natural language**, written with precision, structure, and detail — describing goals, constraints, edge cases, and intent;
- **Existing specification documents** — prior design documents, architecture decision records, regulatory requirements, or organisational standards that can be referenced or attached;
- **References to current systems** — existing data schemas, APIs, software interfaces, and data assets that the solution must interact with or conform to;
- **Semi-formal artefacts** for more technical authors: JSON or YAML configurations, typed schemas, mathematical equations, ontologies, functional specifications, and structured descriptions of non-functional requirements (NFR) such as latency budgets, throughput targets, security constraints, and compliance obligations.

The key principle is that **more structure always helps**. A natural language description is a valid starting point; the same description augmented with typed schemas, NFR budgets, and references to existing systems is a substantially better one. The human is not expected to achieve formal precision — that is the machine's responsibility in the next stage — but they are expected to bring as much clarity and structure as they can.

Stage 2: AI compilation to a formal dependent-typed specification

The first AI compilation step transforms the human problem specification into a formal, dependent-typed machine specification. This is not a trivial translation. It requires the AI to resolve ambiguities, infer missing constraints, and make the implicit explicit. The output is a dependent-typed specification that is:

- **Precise**: every input, output, invariant, and constraint is formally stated;
- **Verifiable**: the typechecker can confirm internal consistency;
- **Inspectable**: a human engineer can read, audit, and edit it before compilation proceeds;
- **Machine-actionable**: it provides an unambiguous target for the subsequent implementation synthesis step.

This intermediate representation is the linchpin of the entire system. Without it, there is no principled mechanism to verify that the human's intent has been correctly

understood. With it, correctness ceases to be a matter of interpretation — it becomes a mathematical property.

The AI iterates with the human at this stage to confirm that the typed specification faithfully captures the intended problem. Ambiguities are surfaced and resolved in dialogue before any implementation is generated. This iteration loop between human intent and machine formalisation is where much of the real value of the pipeline lies.

Stage 3: AI compilation to implementation

The second AI compilation step takes the formal dependent-typed specification and synthesises a working implementation — a program that provably inhabits the specified type. This is type-directed program synthesis: the AI searches the space of programs for one whose type matches the specification, and the typechecker verifies the result with mathematical certainty.

The full pipeline is therefore:

Human spec	$\xrightarrow{\text{AI}}$	**Typed machine spec**	$\xrightarrow{\text{AI}}$	**Implementation**
Natural language, semi-formal artefacts, system references		Dependent types, NFR constraints, formal invariants		Verified program inhabiting the specified type

At each stage, **human intervention is optional but meaningful**. A technically sophisticated engineer may inspect and edit the dependent-typed machine specification — tightening constraints, adding invariants, or correcting a misunderstood requirement — before compilation proceeds to implementation. An engineer may similarly inspect and edit the generated implementation. Crucially, **every edit is checked for consistency against the dependent-typed specification**: the typechecker enforces that no manual modification can introduce a violation of the formally stated requirements. Correctness integrity is maintained throughout.

Why the intermediate specification is essential

One might ask: why not compile directly from the human problem description to the implementation, bypassing the intermediate typed specification? The answer is that without the formal intermediate representation, the system has no principled mechanism to verify that it has correctly understood the human's intent. Natural language is ambiguous; informal artefacts are incomplete; references to existing systems carry implicit assumptions. A direct compilation to code produces something that may look correct but carries no formal guarantee.

The dependent-typed intermediate specification makes the AI's interpretation of the human's intent *explicit and auditable*. It is the point in the pipeline where intent

becomes verifiable fact. Removing it would be analogous to removing the type system from a programming language: the code might still run, but the guarantees disappear.

Why the human specification is the right starting point

Conversely, one might ask: why not require all practitioners to write dependent types directly, eliminating the first compilation step? The answer is that the population of people who possess essential problem knowledge is far larger than the population of type theorists. Domain experts, product engineers, architects, and data scientists all carry knowledge that is indispensable to a correct specification. Requiring type-theoretic literacy as a precondition would recreate the accessibility barrier that kept formal methods from achieving adoption for decades.

The two-stage pipeline democratises access to formal verification. The human contributes *knowledge and intent*; the machine contributes *precision and proof*. Neither can substitute for the other.

The Emerging Stack

Assembling these pieces, we can sketch the future programming stack clearly:

1. **Human writes a structured problem specification**, in natural language augmented with whatever semi-formal artefacts they can bring — typed schemas, NFR constraints, references to existing systems and data. Clarity and structure are the primary virtues at this stage.
2. **AI compiles the human specification into a dependent-typed machine specification**, iterating with the human to resolve ambiguities and confirm that intent has been correctly formalised. This is the new "source of truth" for the system.
3. **AI synthesises a verified implementation** from the typed specification — a program that provably inhabits the specified type. The typechecker verifies correctness with mathematical certainty, replacing the testing and debugging loop for all properties expressible in the specification.
4. **Humans optionally inspect and edit** either the typed specification or the implementation, with the typechecker enforcing consistency at every step.

This is already a live research direction. Program synthesis from types, type-directed code generation using large language models, and AI-assisted formal verification are all active areas at the intersection of programming languages theory and machine learning.

Why Type Theory Is the Right Foundation

Three properties make dependent type theory the natural mathematical home for the problem specification language, rather than natural language or informal diagrams alone.

Compositionality. Types compose. A complex system specification is built from smaller typed components, each independently verifiable, in exactly the way Church envisioned functions composing. This is the direct lineage from lambda calculus to the future of programming.

NFR expressiveness. Refinement types and dependent types make non-functional requirements — latency bounds, memory limits, security invariants, compliance properties — first-class constraints in the specification rather than afterthoughts in a Confluence page. They are verified by the typechecker with the same rigour as functional correctness.

AI alignment. A dependently typed specification gives a coding agent an unambiguous, machine-checkable target. The agent is not guessing intent from a vague prompt; it is searching a well-defined space of programs for one that satisfies a formal proposition. The typechecker closes the loop with certainty rather than probability.

Conclusion

The through-line from Church to here is remarkably clean.

Church said a computation is a function. Martin-Löf said a function's type is a proposition. Curry and Howard said a program is a proof. The vision articulated in this paper says: the human states the proposition, the AI formalises it, and the AI finds the proof.

The industry spent seventy years getting better at the *how*. The next era will be defined by getting rigorous about the *what*. The "new programming language" is, in its mathematical essence, a dependently typed specification language — one that most humans will approach through natural language and semi-formal artefacts, and that AI will compile first into formal precision and then into verified code.

The future of programming is not less rigorous. It is more rigorously focused on the right thing.